

Università degli Studi di Napoli Federico II
Laurea Triennale in Informatica

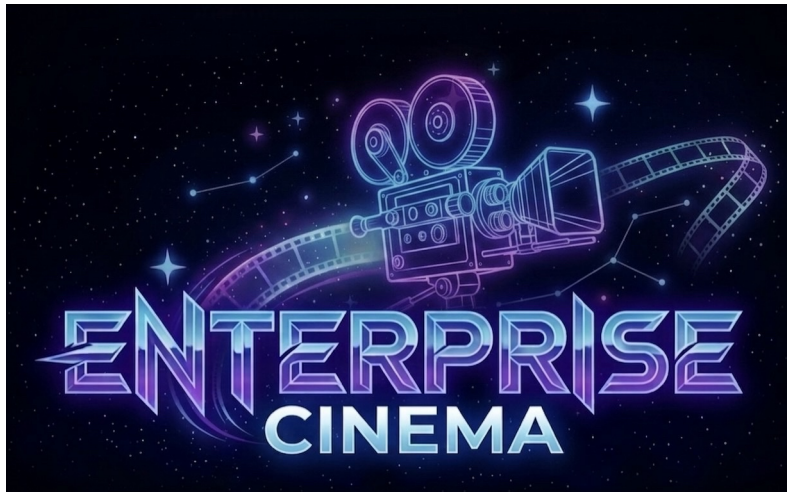
RELAZIONE PROGETTO PROGRAMMAZIONE OBJECT ORIENTED

EnterpriseCinema

Sistema per la Gestione di un Cinema Multisala

Savona Salvatore (DE1000064)
Università degli Studi di Napoli Federico II

Salvato Francesca (N86005865)
Università degli Studi di Napoli Federico II



Indice

I	Introduzione e Requisiti del Sistema	2
I.A	Presentazione del dominio scelto	2
I.B	Scopo del progetto	2
I.C	Requisiti funzionali (Casi d'Uso)	2
I.D	Requisiti di Progettazione Object-Oriented	3
II	Diagramma delle Classi e Presentazione del Dominio scelto	4
II.A	Class Diagram	4
II.B	Classi del Dominio	4
II.C	Associazioni e Cardinalità	6
II.D	Regole di Business	6
III	Lista delle Funzioni ad Alto Livello	8
III.A	Modulo 1: Gestione Utenti ed Autenticazione	8
III.B	Modulo 2: Gestione Film e Programmazione	8
III.C	Modulo 3: Gestione Acquisti, Biglietti e Feedback	9
III.D	Modulo 4: Gestione Segnalazioni e Sale	10
IV	Architettura Software e Struttura dei Package	12
IV.A	BCE + DAO Pattern	12
IV.B	Singleton Pattern per la Persistenza	12
IV.C	Incapsulamento e Disaccoppiamento	13
V	Gestione delle Eccezioni	14
V.A	Gestione su più Livelli	14
V.B	Eccezioni Personalizzate	14
VI	Sequence Diagram delle Operazioni Critiche	15
VI.A	Operazione 1: confermaAcquistoCarrello	15
VI.B	Operazione 2: validaBigliettoPerCodice	17
VII	Repository GitHub	19

I Introduzione e Requisiti del Sistema

I.A Presentazione del dominio scelto

Questo progetto si focalizza nella gestione di un cinema multisala chiamato “Enterprise”. L’obiettivo è quello di coordinare le attività quotidiane della struttura: la programmazione delle proiezioni nelle varie sale, la visualizzazione del catalogo dei film disponibili, la vendita dei biglietti ai clienti, la gestione del carrello per l’acquisto dei titoli e la segnalazione di eventuali problemi o guasti nella struttura. Il cinema Enterprise dispone di due tipologie di sale differenti, sale Standard e sale di tipo IMAX, con prezzi e capienze differenti. Chi utilizza l’applicazione ha la possibilità di accedere come Cliente oppure come Dipendente, e avrà funzionalità diverse a seconda del suo ruolo.

I.B Scopo del progetto

Creare un programma che digitalizzi le richieste e le necessità di clienti e dipendenti. Nello specifico, il programma permette di:

- **Il Cliente** può registrarsi/autenticarsi, visualizzare il catalogo dei film disponibili, aggiungere biglietti per una o più proiezioni al carrello, inserire codici sconto, effettuare un eventuale pagamento, visualizzare i biglietti acquistati e lasciare recensioni ai film (da 1 a 5 stelle).
- **Il Dipendente (Manager, Cassiere, Proiezionista)** può inserire o modificare i film e le proiezioni (decidendo sala, data e orario), convalidare i biglietti all’ingresso utilizzando il codice univoco presente su ogni titolo, segnalare eventuali guasti nelle sale e visualizzare ulteriori segnalazioni riportate da altri membri dello staff (ad esempio proiettore difettoso o sedia guasta).

I.C Requisiti funzionali (Casi d’Uso)

Il sistema permette di fare queste operazioni principali:

1. Registrare e far accedere gli utenti dividendo tra clienti e dipendenti.
2. Modificare le credenziali utente (ai dipendenti non è concesso modificare la mail)
3. Gestire il catalogo dei film (aggiungere, modificare o cancellare).
4. Assegnare i film alle sale e programmare le proiezioni.
5. Inserimento nel carrello di uno o più titoli per una o più proiezioni, scegliendo data, orario e quantità dei biglietti desiderati.
6. Acquistare biglietti simulando un pagamento reale, con eventuali codici sconto (ENTERPRISE, JUNIOR, SENIOR).
7. Generare un codice univoco casuale per ogni biglietto a 8 cifre.
8. Convalidare i biglietti all’ingresso cambiando il loro stato in ‘CONVALIDATO’.
9. Se il biglietto non è stato precedentemente convalidato richiedere un rimborso del titolo eliminandolo dal sistema.

10. Scrivere recensioni con votazione da 1 a 5 e commento di testo.
11. Segnalare e risolvere problemi o guasti nelle diverse sale.

I.D Requisiti di Progettazione Object-Oriented

Il progetto è stato realizzato in linguaggio Java seguendo i principi della programmazione object-oriented. Sfruttando il pattern **BCE (Boundary-Control-Entity)** per la stesura del codice e il pattern **DAO** per memorizzare e leggere i dati dal database. Nello specifico:

- **Incapsulamento:** La configurazione degli attributi delle classi è stata ideata sfruttando diversi livelli appositi di visibilità (`private`, `protected`, `public`). In modo che questi ultimi possano essere letti o modificati esclusivamente tramite l'utilizzo degli appositi metodi `getter` e `setter`.
- **Ereditarietà:** Creazione di una classe astratta `Utente` contenente, che estende le classi `Cliente` e `Dipendente`.
- **Separazione dei ruoli:** Abbiamo diviso la grafica (`gui`), la logica di controllo (`controller`), i dati (`model`) e il codice database (`dao`).

II Diagramma delle Classi e Presentazione del Dominio scelto

II.A Class Diagram

Lo schema concettuale rappresenta il dominio applicativo in formato astratto.

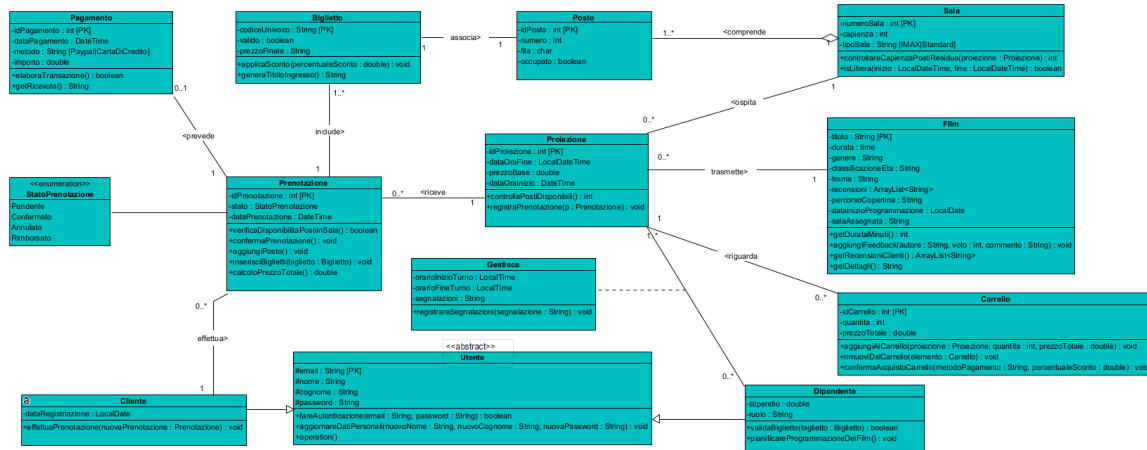


Figura 1: Diagramma delle Classi UML.

II.B Classi del Dominio

Nel package model sono state tradotte le classi dallo schema concettuale, rappresentando le entità portanti del sistema:

1. **Utente (Classe Astratta):** Rappresenta una persona registrata nel database del sistema.

- *Attributi:* Nome, Cognome, email (chiave primaria), password.
- *Metodi:* fareLAutenticazione() per verificare che email e password siano corrette; aggiornareDatiPersonali() per modificare il profilo.

2. **Cliente (Sottoclasse di Utente):** Acquirente del cinema.

- *Attributi:* dataRegistrazione.
- *Metodi:* effettuaPrenotazione() per l'acquisto di nuovi titoli.

3. **Dipendente (Sottoclasse di Utente):** staff del cinema.

- *Attributi:* ruolo (Manager, Cassiere, Proiezionista), stipendio (calcolato nel costruttore dinamicamente in base al ruolo assegnato).
- *Metodi:* pianificareProgrammazioneDeiFilm(). validaBiglietto() per convalidare i titoli dei clienti.

4. **Film:** Contiene i dettagli di una specifica pellicola.

- *Attributi:* titolo, durata, genere, classificazioneEta, trama, recensioni (ArrayList), percorsoCopertina, dataInizioProgrammazione, salaAssegnata.

- *Metodi:* `getDurataMinuti()` per convertire l'orario in minuti; `aggiungiFeedback()` per inserire una recensione; `getDettagli()`, `getRecensioniClienti()`.
5. **Sala:** Rappresenta la sala fisica del cinema.
- *Attributi:* `numeroSala`, `capienza`, `tipoSala` (Standard o IMAX. Sala IMAX 80 posti, Standard 180 posti).
 - *Metodi:* `controllareCapienzaPostiResidua()` per sapere i posti liberi rimanenti; `isLibera()` per controllare che non ci siano altri spettacoli in contemporanea nella stessa sala.
6. **Posto:** Rappresenta la singola sedia in sala.
- *Attributi:* `numero`, `fila`, `occupato`.
7. **Proiezione:** Rappresenta un singolo spettacolo presente nel catalogo (film + sala + orario).
- *Attributi:* `dataOraInizio`, `dataOraFine`, `prezzoBase`.
 - *Metodi:* `controllaPostiDisponibili()`, `registraPrenotazione()`.
8. **Prenotazione:** Rappresenta la procedura di acquisto svolta da un cliente.
- *Attributi:* `dataPrenotazione`, `stato`.
 - *Metodi:* `verificaDisponibilitaPostiInSala()` per controllare la disponibilità della capienza residua in sala, `confermaPrenotazione()` per confermare l'acquisto; `calcoloPrezzoTotale()`, `aggiungiPosto()`, `inserisciBiglietti()`.
9. **Biglietto:** Il singolo titolo d'ingresso acquistato dal cliente.
- *Attributi:* `prezzoFinale`, `valido` (indica se è stato convalidato o meno), `codiceUnivoco` (codice casuale di 8 cifre).
 - *Metodi:* `applicaSconto()` per ridurre il prezzo in caso venga inserito un codice sconto valido; `generaTitoloIngresso()`.
10. **Pagamento:** Registra il pagamento avvenuto.
- *Attributi:* `importo`, `dataPagamento`, `metodo` (PayPal, Carta di Credito).
 - *Metodi:* `elaboraTransazione()`, `getRicevuta()`.
11. **Carrello:** Entità temporanea per memorizzare i titoli selezionati prima di procedere al pagamento.
- *Attributi:* `quantita`, `prezzoTotale`.
 - *Metodi:* `aggiungiAlCarrello()`, `rimuoviDalCarrello()`, `confermaAcquistoCarrello()`.
12. **Gestisce (Classe Associativa):**
- *Attributi:* `orarioInizioTurno`, `orarioFineTurno`, `segnalazioni`.
 - *Metodi:* `registrareSegnalazioni()` per segnalare eventuali guasti e problematiche.

II.C Associazioni e Cardinalità

- **Generalizzazione Utente:** **Utente** si specializza in **Cliente** e **Dipendente**. Si tratta di una generalizzazione totale (tutti gli utenti appartengono ad una delle categorie) ed esclusiva (un utente non può essere contemporaneamente cliente e dipendente).
- **Cliente - Prenotazione** (1 a 0..*): un cliente può effettuare zero o più prenotazioni nel tempo. Ciascuna prenotazione appartiene ad un unico cliente.
- **Prenotazione - Biglietto** (1 a 1..*): una prenotazione contiene uno o più biglietti acquistati per uno stesso checkout.
- **Prenotazione - Pagamento** (1 a 0..1): una prenotazione ha al massimo un pagamento ad essa associato.
- **Biglietto - Posto** (1 a 1): ciascun biglietto è associato ad un unico posto fisico specifico.
- **Biglietto - Proiezione** (1 a 1): ciascun biglietto si riferisce ad una specifica proiezione.
- **Film - Proiezione** (1 a 0..*): un film può avere più proiezioni pianificate. Ciascuna proiezione si riferisce ad un unico film.
- **Sala - Proiezione** (1 a 0..*): una sala ospita più proiezioni durante la giornata. Ciascuna proiezione avviene in un'unica sala.
- **Posto - Sala** (molti a 1): molti posti compongono una sala fisica.
- **Proiezione - Carrello** (1 a 0..*): Una proiezione può essere presente in zero o molti elementi del carrello. Un elemento del Carrello è legato esattamente a una e una sola Proiezione

II.D Regole di Business

Queste sono le regole logiche del cinema da rispettare per un corretto funzionamento dell'applicazione:

1. **Posto unico in sala:** Lo stesso posto (fila e numero) non può essere venduto a due persone diverse per la stessa proiezione.
2. **Limiti recensione:** Il voto di un film deve essere compreso tra 1 e 5.
3. **Rimborso ammesso:** Si può chiedere il rimborso di un biglietto solo se questo non è ancora stato convalidato.
4. **Convalida biglietto:** Solo il dipendente può convalidare un biglietto d'ingresso all'arrivo del cliente sfruttando il codice univoco presente su ogni titolo.
5. **Capienza massima:** Non si possono vendere più biglietti dei posti disponibili in sala per uno spettacolo.

6. **Coerenza della sala:** La sala indicata sul biglietto deve essere la stessa dove il film è effettivamente proiettato a quell'ora.
7. **Applicazione sconti:** Se il cliente usa un codice (ENTERPRISE, SENIOR, JUNIOR), viene applicato un determinato sconto.
8. **Tariffa sala:** I biglietti per gli spettacoli in sala IMAX hanno un sovrapprezzo(12.00€) rispetto ai biglietti per gli spettacoli in sala Standard (8.00€).

III Lista delle Funzioni ad Alto Livello

Le funzioni ad alto livello sono raggruppate in quattro moduli logici:

III.A Modulo 1: Gestione Utenti ed Autenticazione

- **aggiungiCliente:**
 - *Descrizione:* Registra un nuovo account cliente nel database relazionale.
 - *Input:* Oggetto **Cliente** contenente email, nome, cognome, password e data di registrazione.
 - *Output:* **void** (inserimento eseguito).
- **validaLogin:**
 - *Descrizione:* Verifica la corrispondenza delle credenziali inserite dall'utente per l'autenticazione.
 - *Input:* **email** (String), **password** (String).
 - *Output:* **boolean** (esito dell'autenticazione).
- **recuperaUtente:**
 - *Descrizione:* Cerca e restituisce i dati di un utente (Cliente o Dipendente) ricercandolo per indirizzo email.
 - *Input:* **email** (String).
 - *Output:* Oggetto **Utente**.
- **modificaCredenzialiCliente:**
 - *Descrizione:* Permette ad un cliente di aggiornare il proprio indirizzo email di accesso e la password.
 - *Input:* Oggetto **Cliente**, **nuovaEmail** (String), **nuovaPassword** (String).
 - *Output:* **void** (aggiornamento eseguito).
- **modificaPasswordDipendente:**
 - *Descrizione:* Consente a un dipendente di aggiornare la propria password di accesso.
 - *Input:* Oggetto **Dipendente**, **nuovaPassword** (String).
 - *Output:* **void**.

III.B Modulo 2: Gestione Film e Programmazione

- **aggiungiFilm:**
 - *Descrizione:* Inserisce una nuova pellicola all'interno del catalogo del cinema (riservata ai Manager e ai Proiezionisti).
 - *Input:* Oggetto **Film** (titolo, durata, genere, classificazione età, trama, copertina, data inizio programmazione, sala assegnata).

- *Output*: void.
- **modificaFilm**:
 - *Descrizione*: Aggiorna i dettagli di un film già esistente nel catalogo e nel database.
 - *Input*: Oggetto **Film** attuale, e i nuovi parametri (titolo, durata, genere, classificazione, trama, percorso copertina, data inizio, sala assegnata).
 - *Output*: void.
- **eliminaFilm**:
 - *Descrizione*: Rimuove un film dal catalogo cancellandolo fisicamente dal database.
 - *Input*: Oggetto **Film**.
 - *Output*: void.
- **getProiezioniPerFilm**:
 - *Descrizione*: Recupera dal database le proiezioni e gli spettacoli attivi pianificati per il film specificato.
 - *Input*: Oggetto **Film**.
 - *Output*: **ArrayList<Proiezione>**.

III.C Modulo 3: Gestione Acquisti, Biglietti e Feedback

- **acquistaBiglietto**:
 - *Descrizione*: Genera un nuovo biglietto associandogli un posto fisico e una proiezione, lo aggiunge alla memoria di sessione e ne garantisce la persistenza nel database tramite il relativo DAO.
 - *Input*: **prezzo** (double), **posto** (Posto), **proiezione** (Proiezione), **prenotazione** (Prenotazione).
 - *Output*: Oggetto **Biglietto** (il titolo d'ingresso appena generato e registrato).
- **confermaAcquistoCarrello**:
 - *Descrizione*: Esegue la transazione di checkout del carrello, assegna i posti e crea i biglietti calcolando eventuali sconti.
 - *Input*: **metodoPagamento** (String), **percentualeSconto** (double).
 - *Output*: void (acquisti registrati e carrello svuotato).
- **valutaCodiceSconto**:
 - *Descrizione*: Verifica la validità di un codice promozionale inserito in fase di acquisto e restituisce la percentuale di sconto applicabile.
 - *Input*: **codice** (String).
 - *Output*: double (percentuale di sconto).

- **convalidaBigliettoPerCodice:**
 - *Descrizione:* Convalida all'ingresso lo spettatore tramite inserimento del codice univoco del biglietto.
 - *Input:* `codiceUnivoco` (String).
 - *Output:* Oggetto `Biglietto` (stato aggiornato a "CONVALIDATO").
- **rimborsaSingoloBiglietto:**
 - *Descrizione:* Rimuove dal database un biglietto acquistato liberando il posto in sala.
 - *Input:* Oggetto `Biglietto`.
 - *Output:* `void`.
- **aggiungiRecensioneAFilm:**
 - *Descrizione:* Associa ad un film il feedback (voto e commento) rilasciato da un cliente.
 - *Input:* `film` (Film), `cliente` (Cliente), `voto` (int), `commento` (String).
 - *Output:* `boolean` (esito inserimento).
- **ottieniRecensioniDalDB:**
 - *Descrizione:* Recupera in tempo reale dal database tutte le recensioni associate ad un film specifico.
 - *Input:* `titoloFilm` (String).
 - *Output:* `ArrayList<String>` (lista delle recensioni formattate).

III.D Modulo 4: Gestione Segnalazioni e Sale

- **aggiungiSegnalazione:**
 - *Descrizione:* Registra una nuova segnalazione generica nel sistema inviata da un dipendente.
 - *Input:* `messaggio` (String), `mittente` (Dipendente).
 - *Output:* `void`.
- **getSegnalazioni:**
 - *Descrizione:* Recupera lo storico completo delle segnalazioni presenti nel database.
 - *Input:* Nessuno.
 - *Output:* `ArrayList<String>`.
- **verificaStatoSala:**
 - *Descrizione:* Controlla se è presente un guasto o un problema irrisolto per una specifica sala.

- *Input*: `nomeSala` (String).
- *Output*: `String` (messaggio del problema o null se la sala è operativa).
- `segnalaSalaGuasta`:
 - *Descrizione*: Inserisce nel database una segnalazione mirata relativa a un problema tecnico per una sala specifica.
 - *Input*: `mittente` (Dipendente), `nomeSala` (String), `messaggio` (String).
 - *Output*: `void`.
- `riparaSala`:
 - *Descrizione*: Aggiorna lo stato di una segnalazione relativa a una sala impostandola come risolta.
 - *Input*: `nomeSala` (String).
 - *Output*: `void`.

IV Architettura Software e Struttura dei Package

IV.A BCE + DAO Pattern

Per mantenere la modularità del codice, il progetto viene diviso in tre livelli logici separati (BCE), usando in più le interfacce DAO per i collegamenti a livello di database:

- **Package gui (Boundary):** Gestisce la parte grafica del sistema, quello che l'utente vede a schermo. È scritta usando SwingUI designer (es. `Home.java`, tutte le entità `Dashboard...`). I pulsanti e i campi di testo non comunicano mai direttamente con il database, ma passano sempre attraverso **Controller**.
- **Package controller (Control):** Contiene la classe `Controller.java`. È la classe che funge da mediatore all'interno del programma: applica le logiche apposite, riceve le richieste della grafica, fa i controlli di sicurezza e chiama i DAO per salvare o leggere dati.
- **Package model (Entity):** Contiene le classi effettive dei dati (es. `Biglietto`, `Film`).
- **Package dao e implementazionePostgresDAO (DAO):** Per non vincolare il programma a un database specifico, vengono definite le interfacce nel package `dao` (es. `FilmDAO.java`) che vengono poi implementate concretamente nel package `implementazionePostgresDAO` usando codice SQL e connessioni JDBC.
- **Package database:** Contiene la classe `ConnessioneDatabase.java` che gestisce il collegamento fisico al database.

IV.B Singleton Pattern per la Persistenza

Per evitare l'apertura di molteplici connessioni al database (che causano rallentamento del programma e consumo di memoria), abbiamo usato il pattern **Singleton** nella classe `ConnessioneDatabase`. Questo pattern fa sì che venga creata una sola connessione per tutta l'applicazione, e che venga riutilizzata:

Listing 1: Il Singleton in `ConnessioneDatabase.java`

```
public class ConnessioneDatabase {
    private static ConnessioneDatabase instance;
    public Connection connection = null;

    private String nome = "postgres";
    private String password = "password";
    private String url = "jdbc:postgresql://localhost:5432/Cinema";
    private String driver = "org.postgresql.Driver";

    private ConnessioneDatabase() throws SQLException {
        try {
            Class.forName(driver);
            connection = DriverManager.getConnection(url, nome, password);
        } catch (ClassNotFoundException ex) {
            throw new SQLException("Driver PostgreSQL non trovato!", ex);
        }
    }
}
```

```

    }
}

public static ConnessioneDatabase getInstance() throws SQLException {
    if (instance == null || instance.connection == null ||
        instance.connection.isClosed()) {
        instance = new ConnessioneDatabase();
    }
    return instance;
}

public Connection getConnection() { return connection; }
}

```

IV.C Incapsulamento e Disaccoppiamento

Il sistema applica l'incapsulamento impedendo la manipolazione diretta del database o delle liste in memoria. I DAO incapsulano le query SQL parametriche (`PreparedStatement`), impedendo vulnerabilità di tipo SQL Injection. Il controller mantiene in memoria una lista specchio degli oggetti correntemente attivi (es. `listaFilm`, `listaCarrello`) sincronizzandola periodicamente con il DB, riducendo le letture ridondanti.

V Gestione delle Eccezioni

V.A Gestione su più Livelli

Per fare in modo che l'applicazione risulti sicura e robusta, le eccezioni vengono gestite dividendo i ruoli tra i vari package. Quando avviene un errore nel database (es. mancanza del collegamento o inserimento di una chiave duplicata), il codice JDBC solleva una `SQLException`.

1. Il package di implementazione del database propaga l'eccezione marcando il metodo con `throws SQLException`.
2. Il `Controller` cattura questo errore e lancia una nuova eccezione con un messaggio leggibile.
3. La classe della grafica (`gui`) cattura quest'ultimo messaggio e lo mostra a schermo all'utente tramite la finestra `JOptionPane.showMessageDialog`.

V.B Eccezioni Personalizzate

Per gestire le situazioni specifiche del nostro cinema, abbiamo creato 5 classi di eccezione personali nel package `exception`:

- **`PasswordErrataException (Exception)`**: Sollevata se l'utente sbaglia a digitare la password al login.
- **`RecensioneVuotaException (Exception)`**: Sollevata se un cliente prova a salvare una recensione senza scrivere un commento.
- **`SalaPienaException (RuntimeException)`**: Sollevata al pagamento se i posti richiesti superano la capienza della sala.
- **`UtenteNonTrovatoException (Exception)`**: Sollevata se l'email inserita non corrisponde a nessun account registrato.

VI Sequence Diagram delle Operazioni Critiche

In questa sezione vengono descritte due operazioni critiche del sistema, illustrando i rispettivi sequence diagram multi-livello (Controller/GUI, DAO e Modello) e lo pseudocodice dell'operazione.

VI.A Operazione 1: confermaAcquistoCarrello

Questa operazione finalizza l'acquisto dei biglietti presenti nel carrello per il cliente loggato, calcolando gli sconti e allocando fisicamente i posti in sala.

- **Signature:** `public void confermaAcquistoCarrello(String metodoPagamento, double percentualeSconto) throws SalaPienaException`
- **Logica di Business:** Il controller legge gli elementi nel carrello, cerca i posti liberi vicini se disponibili con funzione `trovaPostiVicini()`, calcola i prezzi applicando eventuali maggiorazioni per le sale IMAX e codici sconto, crea gli oggetti prenotazione e biglietto e li inserisce nel database tramite DAO.
- **Operazioni DB:** Inserimento di record nella tabella `biglietto` tramite `BigliettoDAO.inserisci`.

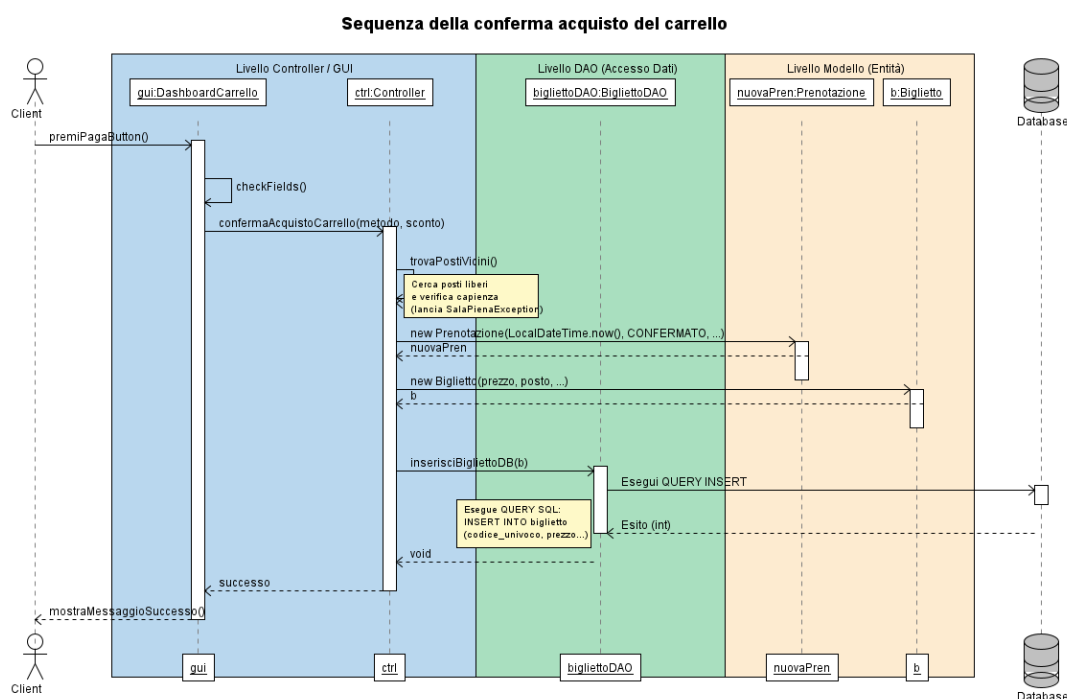


Figura 2: Diagramma di sequenza dell'operazione confermaAcquistoCarrello.

Listing 2: Pseudocodice dell'operazione confermaAcquistoCarrello

```
prezzoBase = 8.00
prezzoSingoloScontato = prezzoBase - (prezzoBase * (percentualeSconto / 100.0))

PER OGNI ElementoCarrello elem IN carrello DO
```



```

quantita = elem.getQuantita()
proiezione = elem.getProiezione()

IF "IMAX" == elem.getProiezione().getSala().getTipoSala() THEN
    prezzoBase = 12.00
ENDIF

postiAssegnati = trovaPostiVicini(proiezione, quantita) // solleva
    eccezione se piena
nuovaPrenotazione = NEW Prenotazione(LocalDateTime.now(),
    StatoPrenotazione.CONFERMATO, proiezione, utenteLoggatoTemporaneo)

PER i = 0 TO quantita - 1 DO
    postoEsatto = postiAssegnati.get(i)
    b = acquistaBiglietto(prezzoSingoloScontato, postoEsatto, proiezione,
        nuovaPrenotazione)
    nuovaPrenotazione.getBiglietti().add(b)
ROUTINE
ROUTINE
svuotaCarrello()

```

VI.B Operazione 2: validaBigliettoPerCodice

Questa operazione verifica che un biglietto esista e non sia già stato convalidato, aggiornando lo stato nel database.

- **Segnatura:** `public Biglietto convalidaBigliettoPerCodice(String codiceUnivoco) throws Exception`
- **Logica di Business:** Il controller cerca il biglietto usando il codice univoco. Se lo trova e non è ancora stato usato, esegue la convalida tramite la classe Dipendente e aggiorna lo stato sul database.
- **Operazioni DB:** Chiamata a `BigliettoDAO.aggiornaStatoBigliettoDB(codice, "CONVALIDATO")`.

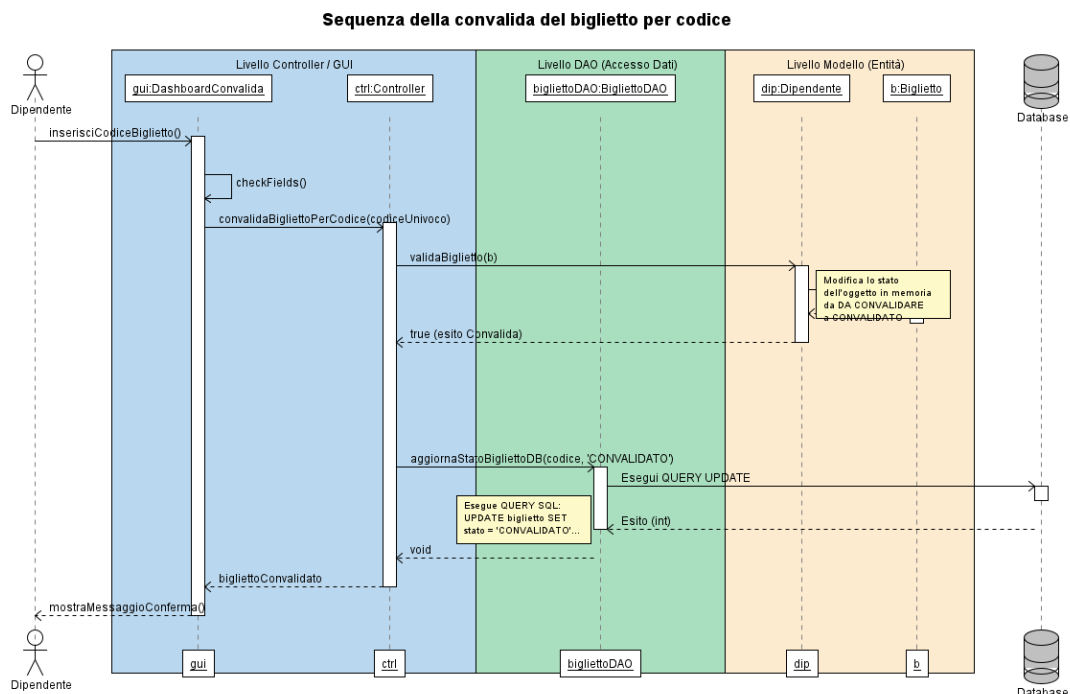


Figura 3: Diagramma di sequenza dell'operazione convalidaBigliettoPerCodice.

Listing 3: Pseudocodice dell'operazione convalidaBigliettoPerCodice

```
// Nel Controller: convalidaBigliettoPerCodice(codiceUnivoco)
IF codiceUnivoco == null OR vuoto THEN THROW Exception("Codice non valido")

PER OGNI biglietto b IN listaBiglietti DO
    IF b.getCodiceUnivoco() == codiceUnivoco THEN
        IF b.isValido() THEN THROW Exception("GIA CONVALIDATO")

        dipendenteGenerico = listaDipendenti.get(0)
        IF NOT dipendenteGenerico.validaBiglietto(b) THEN
            THROW Exception("Biglietto gia convalidato")
        ENDIF
    ENDIF
```

```
        // Operazione DB tramite DAO
        bigliettoDAO.aggiornaStatoBigliettoDB(codiceUnivoco, "CONVALIDATO")
        RETURN b
    ENDIF
ROUTINE
THROW Exception("Codice inesistente")
```

VII Repository GitHub

- Link alla repository del progetto:
<https://github.com/sammyyyy419/Gruppo-Progetto-P00-Savona-Salvato>